

CS3014 - Estructuras de Datos Avanzadas

Orthogonal Range Search

Semana 4

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

2 de septiembre de 2026

¿Qué aprenderemos hoy?



¿Qué aprenderemos hoy?



- Orthogonal range search

¿Qué aprenderemos hoy?



- ▶ Orthogonal range search
- ▶ Range trees

Orthogonal Range Search

Definición



Problema

Definición



Problema

Dados n puntos en $\mathbb{R}^d$, se nos pedirá realizar consultas sobre cajas; es decir, se nos darán d intervalos $[l_i, r_i]$ y obtendremos información de los puntos en $[l_1, r_1] \times [l_2, r_2] \times \dots \times [l_d, r_d]$. Al producto cartesiano de los intervalos de consulta le llamaremos *caja*.

Definición



Problema

Dados n puntos en $\mathbb{R}^d$, se nos pedirá realizar consultas sobre cajas; es decir, se nos darán d intervalos $[l_i, r_i]$ y obtendremos información de los puntos en $[l_1, r_1] \times [l_2, r_2] \times \dots \times [l_d, r_d]$. Al producto cartesiano de los intervalos de consulta le llamaremos *caja*.

Objetivo

Obtener una complejidad de $O(\log n + k)$ por consulta, donde k es el tamaño de la respuesta.

Definición



Problema

Dados n puntos en $\mathbb{R}^d$, se nos pedirá realizar consultas sobre cajas; es decir, se nos darán d intervalos $[l_i, r_i]$ y obtendremos información de los puntos en $[l_1, r_1] \times [l_2, r_2] \times \dots \times [l_d, r_d]$. Al producto cartesiano de los intervalos de consulta le llamaremos *caja*.

Objetivo

Obtener una complejidad de $O(\log n + k)$ por consulta, donde k es el tamaño de la respuesta.

En 2D normalmente es factible dicha complejidad.

Definición



Escenarios

Definición



Escenarios

- ▶ Estático: Los puntos son fijos, así que podrían preprocesarse.

Definición



Escenarios

- ▶ Estático: Los puntos son fijos, así que podrían preprocesarse.
- ▶ Dinámico: Se puede agregar y/o eliminar puntos.

Definición



Escenarios

- ▶ Estático: Los puntos son fijos, así que podrían preprocesarse.
- ▶ Dinámico: Se puede agregar y/o eliminar puntos.

Variantes

Definición



Escenarios

- ▶ Estático: Los puntos son fijos, así que podrían preprocesarse.
- ▶ Dinámico: Se puede agregar y/o eliminar puntos.

Variantes

- ▶ Existencia: ¿Existe algún punto dentro de la caja?

Definición



Escenarios

- ▶ Estático: Los puntos son fijos, así que podrían preprocesarse.
- ▶ Dinámico: Se puede agregar y/o eliminar puntos.

Variantes

- ▶ Existencia: ¿Existe algún punto dentro de la caja?
- ▶ Conteo: ¿Cuántos puntos hay dentro de la caja?

Definición



Escenarios

- ▶ Estático: Los puntos son fijos, así que podrían preprocesarse.
- ▶ Dinámico: Se puede agregar y/o eliminar puntos.

Variantes

- ▶ Existencia: ¿Existe algún punto dentro de la caja?
- ▶ Conteo: ¿Cuántos puntos hay dentro de la caja?
- ▶ Enumeración: ¿Cuáles son todos los puntos dentro de la caja? / ¿Puedes darme k puntos que estén dentro de la caja?

Analicemos dimensiones



Caso $d = 1$

Analicemos dimensiones



Caso $d = 1$

- ▶ La consulta es solo un intervalo.

Analicemos dimensiones



Caso $d = 1$

- ▶ La consulta es solo un intervalo.
- ▶ Podemos ordenar los puntos para responder a las consultas usando búsqueda binaria.

Analicemos dimensiones



Caso $d = 1$

- ▶ La consulta es solo un intervalo.
- ▶ Podemos ordenar los puntos para responder a las consultas usando búsqueda binaria.
- ▶ Tendremos un preprocesamiento de $O(n \log n)$ y consulta de $O(\log n + k)$.

Analicemos dimensiones



Caso $d = 1$

- ▶ La consulta es solo un intervalo.
- ▶ Podemos ordenar los puntos para responder a las consultas usando búsqueda binaria.
- ▶ Tendremos un preprocesamiento de $O(n \log n)$ y consulta de $O(\log n + k)$.

Observación

Analicemos dimensiones



Caso $d = 1$

- ▶ La consulta es solo un intervalo.
- ▶ Podemos ordenar los puntos para responder a las consultas usando búsqueda binaria.
- ▶ Tendremos un preprocesamiento de $O(n \log n)$ y consulta de $O(\log n + k)$.

Observación

Si usamos arreglos, va a ser complicado aumentar dimensiones de manera “anidada”.

Analicemos dimensiones



Idea final

Analicemos dimensiones



Idea final

Podemos usar árboles binarios de búsqueda balanceados que almacenen a los puntos en las hojas.

Analicemos dimensiones



Idea final

Podemos usar árboles binarios de búsqueda balanceados que almacenen a los puntos en las hojas.

Cada nodo que no sea hoja debe almacenar información que permita realizar la búsqueda de un valor en las hojas.

Analicemos dimensiones



Idea final

Podemos usar árboles binarios de búsqueda balanceados que almacenen a los puntos en las hojas.

Cada nodo que no sea hoja debe almacenar información que permita realizar la búsqueda de un valor en las hojas.

Es posible solo almacenar el máximo del subárbol izquierdo para minimizar la memoria usada.

Analicemos dimensiones



Idea final

Podemos usar árboles binarios de búsqueda balanceados que almacenen a los puntos en las hojas.

Cada nodo que no sea hoja debe almacenar información que permita realizar la búsqueda de un valor en las hojas.

Es posible solo almacenar el máximo del subárbol izquierdo para minimizar la memoria usada.

Observación

Analicemos dimensiones



Idea final

Podemos usar árboles binarios de búsqueda balanceados que almacenen a los puntos en las hojas.

Cada nodo que no sea hoja debe almacenar información que permita realizar la búsqueda de un valor en las hojas.

Es posible solo almacenar el máximo del subárbol izquierdo para minimizar la memoria usada.

Observación

El árbol tendrá $O(n)$ nodos y las complejidades se mantienen.

Analicemos dimensiones

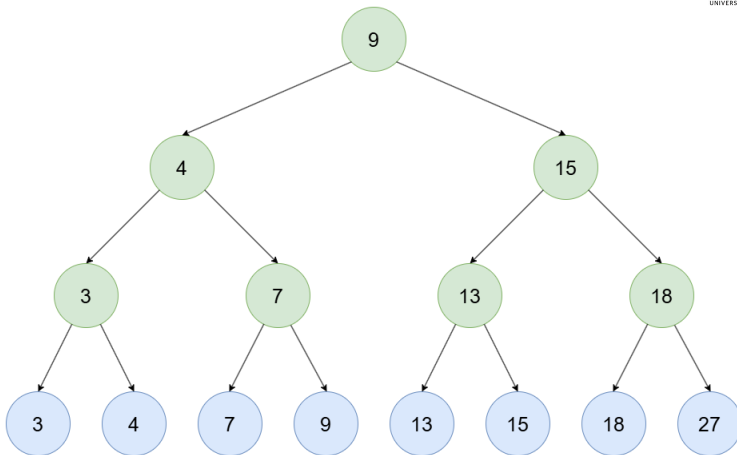


Figura: Range tree de una dimensión

Analicemos dimensiones



Realizando consultas

Analicemos dimensiones



Realizando consultas

Para seleccionar el rango de posiciones que están contenidas en el rango de consulta $[l_1, r_1]$ podemos hacer las consultas $\text{PREDECESSOR}(l_1)$ y $\text{SUCCESSOR}(r_1)$ — Todas las posiciones contenidas entre ese par de nodos serán parte de la respuesta.

Analicemos dimensiones



Realizando consultas

Para seleccionar el rango de posiciones que están contenidas en el rango de consulta $[l_1, r_1]$ podemos hacer las consultas $\text{PREDECESSOR}(l_1)$ y $\text{SUCCESSOR}(r_1)$ — Todas las posiciones contenidas entre ese par de nodos serán parte de la respuesta.

Observación

Analicemos dimensiones



Realizando consultas

Para seleccionar el rango de posiciones que están contenidas en el rango de consulta $[l_1, r_1]$ podemos hacer las consultas $\text{PREDECESSOR}(l_1)$ y $\text{SUCCESSOR}(r_1)$ — Todas las posiciones contenidas entre ese par de nodos serán parte de la respuesta.

Observación

Sea $P = \text{LCA}(\text{PREDECESSOR}(l_1), \text{SUCCESSOR}(r_1))$, entonces para cada nodo en el camino de P a $\text{PREDECESSOR}(l_1)$ que baje por su hijo izquierdo, su hijo derecho pertenece a nuestra respuesta. Se cumple algo análogo en el camino de P a $\text{SUCCESSOR}(r_1)$.

Analicemos dimensiones



Realizando consultas

Para seleccionar el rango de posiciones que están contenidas en el rango de consulta $[l_1, r_1]$ podemos hacer las consultas $\text{PREDECESSOR}(l_1)$ y $\text{SUCCESSOR}(r_1)$ — Todas las posiciones contenidas entre ese par de nodos serán parte de la respuesta.

Observación

Sea $P = \text{LCA}(\text{PREDECESSOR}(l_1), \text{SUCCESSOR}(r_1))$, entonces para cada nodo en el camino de P a $\text{PREDECESSOR}(l_1)$ que baje por su hijo izquierdo, su hijo derecho pertenece a nuestra respuesta. Se cumple algo análogo en el camino de P a $\text{SUCCESSOR}(r_1)$.

Esto quiere decir que tendremos $O(\log n)$ subárboles que contengan nuestra respuesta.

Analicemos dimensiones

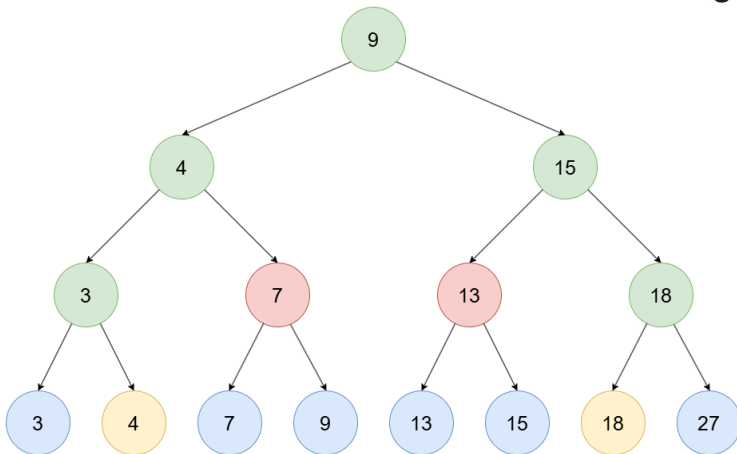


Figura: Ejemplo de búsqueda con $[l_1, r_1] = [5, 16]$. Los nodos amarillos son los nodos delimitadores y los nodos rojos son los subárboles de la respuesta.

Analicemos dimensiones



¿Y como respondemos a cada variante?

Analicemos dimensiones



¿Y como respondemos a cada variante?

- Existencia/Conteo: Podemos hacer que cada nodo almacene la cantidad de hojas en su subárbol y así responder en $O(\log n)$ debido a que tendremos que sumar $O(\log n)$ valores.

Analicemos dimensiones



¿Y como respondemos a cada variante?

- ▶ Existencia/Conteo: Podemos hacer que cada nodo almacene la cantidad de hojas en su subárbol y así responder en $O(\log n)$ debido a que tendremos que sumar $O(\log n)$ valores.
- ▶ Enumeración: Podemos iterar sobre cada uno de los $O(\log n)$ subárboles que nos interesan y detenernos cuando hayamos enumerado la cantidad deseada.

Analicemos dimensiones



¿Y como respondemos a cada variante?

- ▶ Existencia/Conteo: Podemos hacer que cada nodo almacene la cantidad de hojas en su subárbol y así responder en $O(\log n)$ debido a que tendremos que sumar $O(\log n)$ valores.
- ▶ Enumeración: Podemos iterar sobre cada uno de los $O(\log n)$ subárboles que nos interesan y detenernos cuando hayamos enumerado la cantidad deseada.

¿Cómo se llama esta estructura?

Analicemos dimensiones



¿Y como respondemos a cada variante?

- ▶ Existencia/Conteo: Podemos hacer que cada nodo almacene la cantidad de hojas en su subárbol y así responder en $O(\log n)$ debido a que tendremos que sumar $O(\log n)$ valores.
- ▶ Enumeración: Podemos iterar sobre cada uno de los $O(\log n)$ subárboles que nos interesan y detenernos cuando hayamos enumerado la cantidad deseada.

¿Cómo se llama esta estructura?

Esta estructura se llama *Range tree*.

Analicemos dimensiones



¿Y como respondemos a cada variante?

- ▶ Existencia/Conteo: Podemos hacer que cada nodo almacene la cantidad de hojas en su subárbol y así responder en $O(\log n)$ debido a que tendremos que sumar $O(\log n)$ valores.
- ▶ Enumeración: Podemos iterar sobre cada uno de los $O(\log n)$ subárboles que nos interesan y detenernos cuando hayamos enumerado la cantidad deseada.

¿Cómo se llama esta estructura?

Esta estructura se llama *Range tree*.

Además, nos permite extenderla para múltiples dimensiones.

Intentemos implementar la versión en 1D



Static Range Sum

Dada una secuencia a de tamaño n y q consultas (l, r) , para cada consulta debemos calcular la suma de los elementos de a en el rango de posiciones $[l, r)$.

Range tree

Pasando de 1 a 2 dimensiones



Aprovechando el caso $d = 1$

Pasando de 1 a 2 dimensiones



Aprovechando el caso $d = 1$

En el caso $d = 1$ tenemos una estructura que nos permite filtrar toda la información en un rango en $O(\log n)$.

Pasando de 1 a 2 dimensiones



Aprovechando el caso $d = 1$

En el caso $d = 1$ tenemos una estructura que nos permite filtrar toda la información en un rango en $O(\log n)$.

Para $d = 2$, es posible hacer que cada nodo del range tree que usamos para $d = 1$ tenga un puntero hacia range tree sobre las hojas que contiene su subárbol pero usando la componente Y como llave en vez de la X .

Pasando de 1 a 2 dimensiones

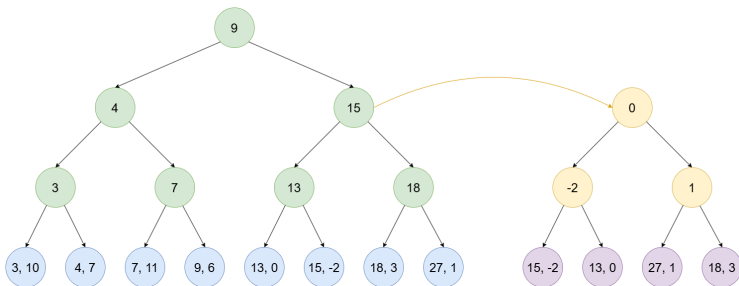


Figura: Range tree de 2 dimensiones. El árbol de la derecha es un range tree sobre los puntos $(13, 0)$, $(15, -2)$, $(18, 3)$, $(27, 1)$ pero con llave Y .

Pasando de 1 a 2 dimensiones



¿Pero eso no es muy costoso?

Pasando de 1 a 2 dimensiones



¿Pero eso no es muy costoso?

Podría parecerlo, pero al ser un árbol binario de búsqueda balanceado, cada hoja debería estar incluida en $O(\log n)$ subárboles, así que la memoria total terminaría siendo $O(n \log n)$.

Pasando de 1 a 2 dimensiones



¿Pero eso no es muy costoso?

Podría parecerlo, pero al ser un árbol binario de búsqueda balanceado, cada hoja debería estar incluida en $O(\log n)$ subárboles, así que la memoria total terminaría siendo $O(n \log n)$.

Además, la consulta para 2 dimensiones se reduce a una consulta de $O(\log n)$ en $O(\log n)$ árboles, así que terminamos con una complejidad de $O(\log^2 n + k)$.

Pasando de 1 a 2 dimensiones



¿Pero eso no es muy costoso?

Podría parecerlo, pero al ser un árbol binario de búsqueda balanceado, cada hoja debería estar incluida en $O(\log n)$ subárboles, así que la memoria total terminaría siendo $O(n \log n)$.

Además, la consulta para 2 dimensiones se reduce a una consulta de $O(\log n)$ en $O(\log n)$ árboles, así que terminamos con una complejidad de $O(\log^2 n + k)$.

¿Y para d más grandes?

Pasando de 1 a 2 dimensiones



¿Pero eso no es muy costoso?

Podría parecerlo, pero al ser un árbol binario de búsqueda balanceado, cada hoja debería estar incluida en $O(\log n)$ subárboles, así que la memoria total terminaría siendo $O(n \log n)$.

Además, la consulta para 2 dimensiones se reduce a una consulta de $O(\log n)$ en $O(\log n)$ árboles, así que terminamos con una complejidad de $O(\log^2 n + k)$.

¿Y para d más grandes?

Podemos plantear una estructura anidada, de manera que en cada uno de los árboles de una componente en la dimensión k podemos agregar punteros hacia range trees ordenados por la dimensión $(k + 1)$.

Pasando a d dimensiones



Complejidad

Pasando a d dimensiones



Complejidad

- ▶ Cada range tree correspondiente a subárboles puede ser construido en $O(n)$ usando la información de sus dos hijos.

Pasando a d dimensiones



Complejidad

- ▶ Cada range tree correspondiente a subárboles puede ser construido en $O(n)$ usando la información de sus dos hijos.
- ▶ Preprocesamiento: $O(n \log^{d-1} n)$ para todo $d \geq 2$ (notemos que para $d = 1$ sí o sí tendremos $O(n \log n)$ por ordenar los puntos según su componente X).

Pasando a d dimensiones



Complejidad

- ▶ Cada range tree correspondiente a subárboles puede ser construido en $O(n)$ usando la información de sus dos hijos.
- ▶ Preprocesamiento: $O(n \log^{d-1} n)$ para todo $d \geq 2$ (notemos que para $d = 1$ sí o sí tendremos $O(n \log n)$ por ordenar los puntos según su componente X).
- ▶ Espacio: $O(n \log^{d-1} n)$.

Pasando a d dimensiones



Complejidad

- ▶ Cada range tree correspondiente a subárboles puede ser construido en $O(n)$ usando la información de sus dos hijos.
- ▶ Preprocesamiento: $O(n \log^{d-1} n)$ para todo $d \geq 2$ (notemos que para $d = 1$ sí o sí tendremos $O(n \log n)$ por ordenar los puntos según su componente X).
- ▶ Espacio: $O(n \log^{d-1} n)$.
- ▶ Consultas: $O(\log^d n + k)$.

Pasando a d dimensiones



Complejidad

- ▶ Cada range tree correspondiente a subárboles puede ser construido en $O(n)$ usando la información de sus dos hijos.
- ▶ Preprocesamiento: $O(n \log^{d-1} n)$ para todo $d \geq 2$ (notemos que para $d = 1$ sí o sí tendremos $O(n \log n)$ por ordenar los puntos según su componente X).
- ▶ Espacio: $O(n \log^{d-1} n)$.
- ▶ Consultas: $O(\log^d n + k)$.

¿Es posible mejorarlo?

Pasando a d dimensiones



Complejidad

- ▶ Cada range tree correspondiente a subárboles puede ser construido en $O(n)$ usando la información de sus dos hijos.
- ▶ Preprocesamiento: $O(n \log^{d-1} n)$ para todo $d \geq 2$ (notemos que para $d = 1$ sí o sí tendremos $O(n \log n)$ por ordenar los puntos según su componente X).
- ▶ Espacio: $O(n \log^{d-1} n)$.
- ▶ Consultas: $O(\log^d n + k)$.

¿Es posible mejorarlo?

Sí, se puede mejorar por un factor de $O(\log n)$ las consultas para $d \geq 2$.

Intentemos implementar la versión en 2D



Rectangle Sum

Intentemos implementar la versión en 2D



Rectangle Sum

Dados n puntos (x, y) con pesos asociados y q consultas (l, d, r, u) , para cada consulta debemos determinar la suma de todos los puntos con $l \leq x < r$ y $d \leq y < u$.

Resumen de la sesión



Resumen de la sesión



- ▶ Aprendimos cómo se plantea la estructura del Range Tree en 1D

Resumen de la sesión



- ▶ Aprendimos cómo se plantea la estructura del Range Tree en 1D
- ▶ Aprendimos cómo se extiende a más dimensiones el Range Tree.

Resumen de la sesión



- ▶ Aprendimos cómo se plantea la estructura del Range Tree en 1D
- ▶ Aprendimos cómo se extiende a más dimensiones el Range Tree.
- ▶ Implementamos un Range Tree en 1D y 2D.

Gracias